



Build Your Own Tool

Custom Tools + Agent Memory

Last week you USED tools and memory. This week, you give your agent its own.

Where We Left Off

Last week you ran a real agent and gave it powers. Remember the three kinds of tools?

Built-in Tools

Read, write, run — came with the agent

MCP Servers

Plug-in packs that add NEW tools

Skills

Saved instructions you wrote

This week: you BUILD your own MCP tool, and give your agent a MEMORY that survives a restart.



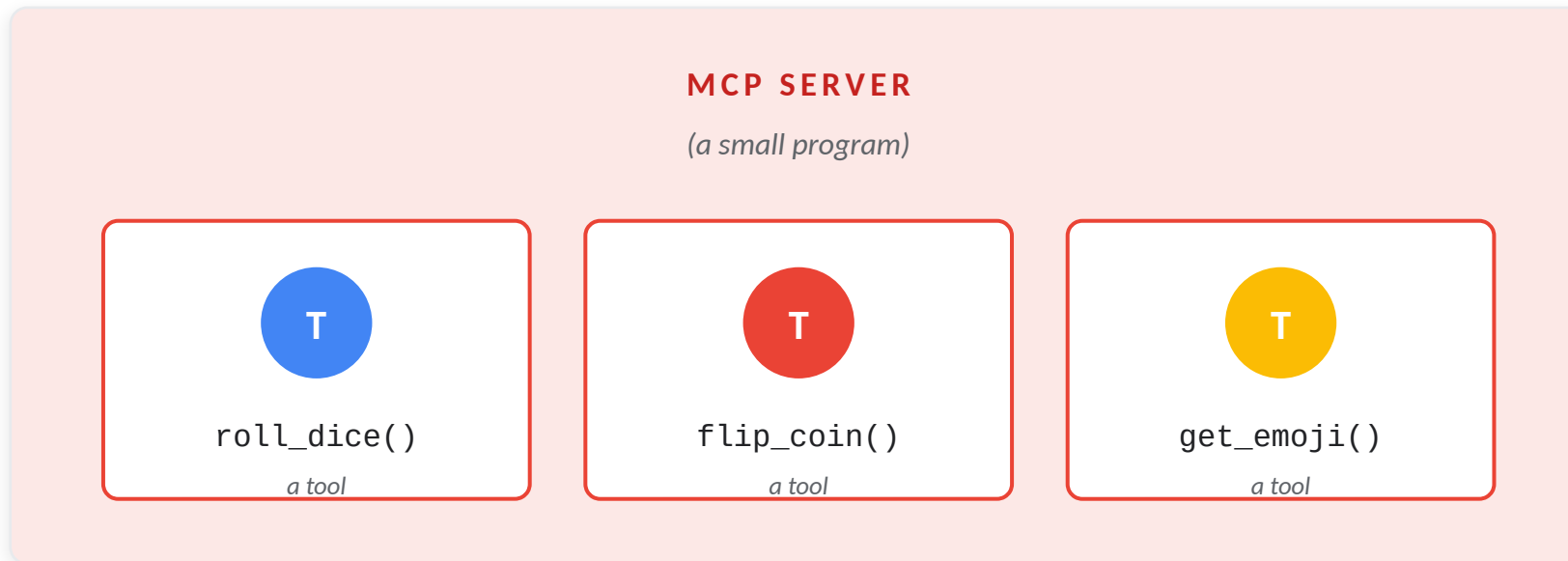
What if your agent could do something **nobody built yet?**

Then you build the tool yourself.

Today you'll have your agent create a brand-new tool, plug it in, and use it — plus give it a memory so it remembers you next time. You're not just driving the agent anymore. You're upgrading it.

What's Inside an MCP Server?

Last week you plugged in MCP servers other people made. But what IS one? It's just a small program that offers one or more tools the agent can call.



Each tool is just one function. Build a server = write some functions and let the agent call them.

A Tool is Just a Function

Here's a real tool. Three parts to notice:

```
@mcp.tool()

def roll_dice(sides):

    """Roll a dice with the

    given number of sides."""

    return random.randint(1, sides)
```

The name

What the tool is called

The inputs

What the agent passes in (sides)

The docstring

What the agent READS to decide when to use it

The Docstring is the Description

The agent can't read your mind — it reads the docstring to decide WHICH tool to use. Clear words win. (Sound familiar?)

VAGUE

```
"""does stuff"""
```

The agent has no idea when to use this tool.

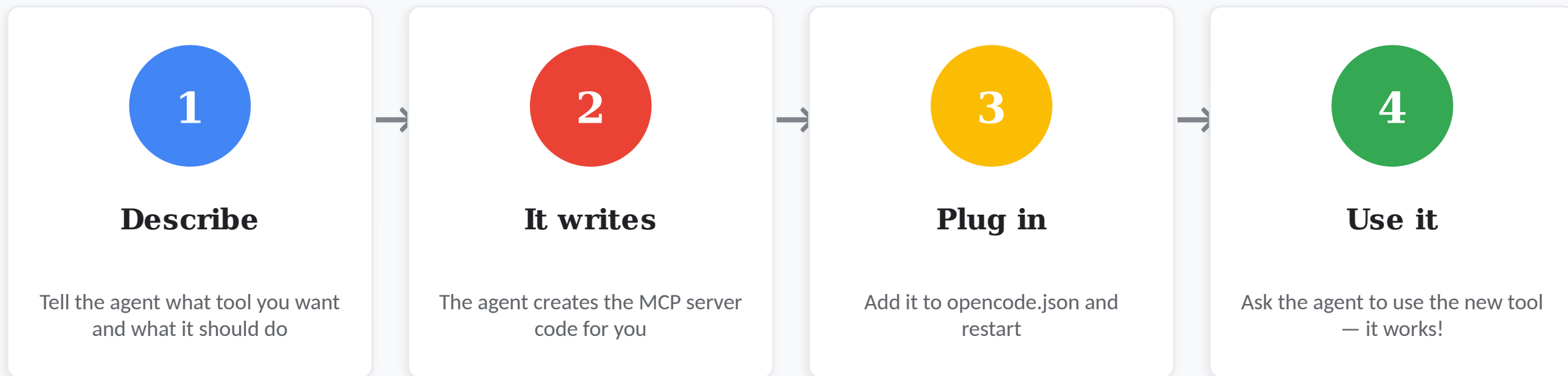
CLEAR

```
"""Roll a dice. Use this  
whenever the user wants a  
random number or to roll."""
```

Now the agent knows exactly when to reach for it.

Let the Agent Build the Tool!

Here's the twist: you don't have to write the Python yourself. Your agent can write its OWN new tool — you just describe it clearly.



You're vibe coding a tool — describe it well, and the agent builds it.

Tools You Could Build



Dice Roller

Roll any number of sides



Coin Flipper

Heads or tails



Emoji Picker

An emoji for any mood



Secret Decoder

Turn text into secret code



Name Generator

Random fantasy names



Leap Year Checker

Is this year a leap year?

Give Your Agent a Memory

By itself, your agent forgets everything when you restart. Last week you learned the types of memory — today you actually give it one.

WITHOUT memory

You: "My name is Sam."

Agent: "Nice to meet you!"

— *restart* —

You: "What's my name?"

Agent: "I don't know!"

WITH memory

You: "Remember my name is Sam."

Agent: writes it to memory

— *restart* —

You: "What's my name?"

Agent: "You're Sam!"

Memory is Just a File

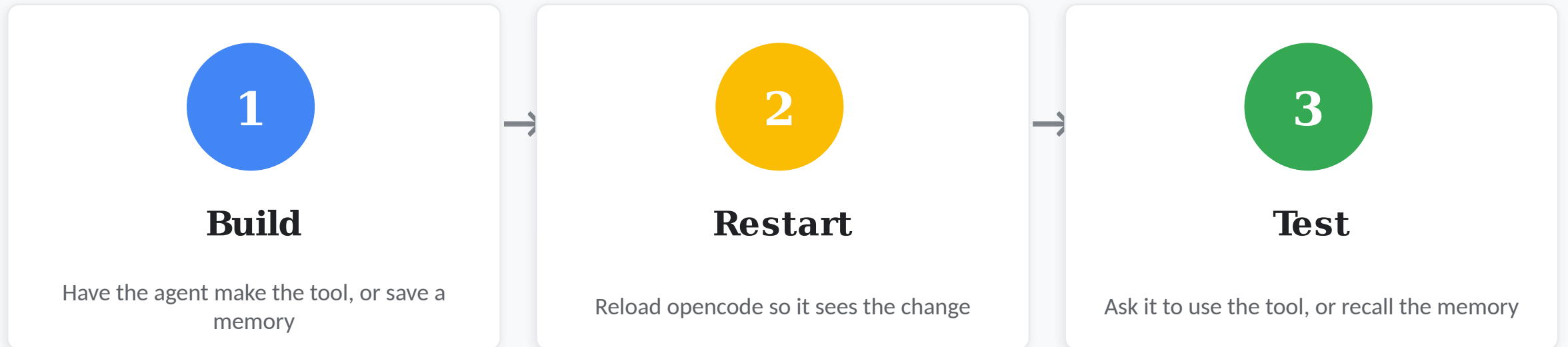
opencode reads a special file called AGENTS.md every time it starts. Whatever's written there, the agent remembers. So memory = the agent writing notes to itself in that file.



That's long-term memory — saved facts the agent looks up across sessions.

Build · Restart · Test

Whether it's a new tool or new memory, the rhythm is the same. Restarting is how the agent picks up the change.



Forgot to restart? The agent won't see your change yet — restart and try again.

Today's Missions

A

Build a Tool

Have your agent create a custom tool

B

Use Your Tool

Plug it in, restart, and try it out

C

Improve It

Upgrade your tool with a new feature

D

Give It Memory

Make it remember you after a restart

E

Interview It

Ask the agent how your upgrades work

F

Quick Quests

Build more tools & save more memories

G

Design & Share

Invent one more, show off your coolest

Tool-Builder Rules



Do

- Write clear docstrings so the agent knows when to use a tool
- Keep tools small and safe (dice, emojis, codes)
- Restart after every change
- Test the tool before trusting it



Don't

- Don't put private info in the memory file
- Don't build tools that delete or change important files
- Don't approve code you don't understand — ask first
- Don't forget: tools run REAL code



Go Upgrade Your Agent

Build a tool nobody's built before • give it a memory of its own.

Head to the classwork and work through Missions A–E with your group. Describe your tool clearly, restart after changes, and test everything. Kahoot at the end!

Last week you directed agents. This week you upgrade them.